xv6-aarch64 tty 子系统：分层与数据通路（rpi3-lock 分支）

用户态

sh / 普通进程
`read(0) write(1) ^C`

系统调用 ttyattach / tcsetpgrp
`sys_ttyattach sys_ttysetfg`

sshd / ptytest：持 master，子进程持 slave
`ptyalloc() → 两个 FD_PTY file`

VFS / 字符设备

`/dev/console CONSOLE`
uart_probe 注册 fops

`/dev/ttyS0 TTYS0`
ttyinit：同一组 read/write

`/dev/tty TTY`
按 p->ctty 转发，非 TTYS0 返回 -1

`pty_file_ops {fread,fwrite,release,poll}`
不走 devsw，file->f_op 直接挂

tty 层（kernel/drivers/tty）

ctty==TTYS0

console.c：简易行规程

🔒 **cons.lock 保护 buf[128] / r / w / e**



`consoleintr(c)` ← 生产者(中断/工作队列)
回显、^H ^U ^D ^P，遇 \n 令 w=e 并 wakeup(&cons.r)
`consoleread()` ← 消费者(进程)
r==w 时 sleep(&cons.r, &cons.lock)
`consolewrite()` 不拿 `cons.lock`
每字节 uartputc()，\n 前补 \r
`consputc()` 回显 / printf 用，同步发送

^C

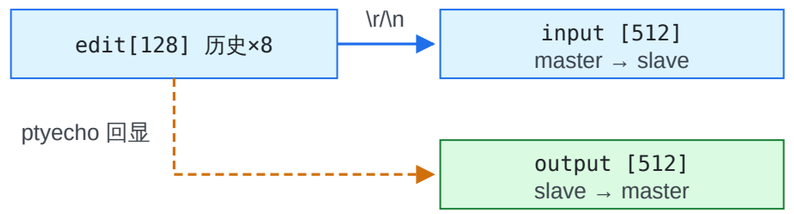
tty.c：会话 / 前台进程组

`ttyattach: sid=pgid=pid, ctty=TTYS0`
`ttysetforeground(pid) ttyintr()`
`static int ttyfg_pgid;`
🔒 **ttyfg_lock（只保护 ttyfg_pgid）**

signal_pgrp(fg, SIGINT)
逐个 🔒 p->lock，置 killed

pty.c：伪终端（每对一个 struct pty）

🔒 **pt->lock 保护两条队列 + 编辑行 + 历史 + open 标志**



`master 写 → ptyinput()：ESC[A/B 历史、退格`
`slave 写 → ONLCR 后进 output`
满则 `sleep(&q->nwrite)`，空则 `sleep(&q->nread)`
每次读/写后 epollnotify_pty() → 🔒 ep->lock

低层驱动

输入：uartintr → consoleintr
write：uartputc（可睡眠）

uart.c：BCM2837 Mini UART

🔒 **uart_tx_lock 保护 uart_tx_buf[32] / tx_w / tx_r**
🔒 **uart_hw_lock 最内层："查 LSR + 写 mu_io" 原子化**
`uartputc` 入环形缓冲，满则 `sleep(&uart_tx_r)`
`uartstart` 搬到 FIFO，FIFO 满则开 TX 中断
`uartputc_sync` push_off + 忙等 FIFO（不排队）
`uartintr` RX 逐字节 `consoleintr`；再 `uartstart`

USB 键盘也注入同一 cons.buf

printf.c

🔒 **pr.lock**
只防多个 printf 交错
`printf → consputc`
panic: pr.locking=0，
panicked=1 冻结其它 CPU

usbkbd.c：USB HID 键盘

🔒 **kbd->lock 只保护 active/irq_urb 等状态**
`usbkbd_irq_complete`（中断）→ `queue_work`
`usbkbd_rx_work`（工作队列，进程上下文）
→ `usbkbd_key` → `consoleintr(c)`
调 consoleintr 前已放掉 kbd->lock，不嵌套

printf/回显：uartputc_sync

硬件 / 中断

Mini UART：mu_io / mu_lsr / mu_ier，8 字节 FIFO，115200 8N1
`GIC UART0_IRQ → devintr() → uartintr()`

DWC2 USB 主控
`DWC2_IRQ → dwc2_irq() → URB 完成回调`

输入通路 | write() 异步输出（经 32 字节环） | printf / 回显同步输出（绕过环） | ^C → 前台进程组 | 🔒 xxx = spinlock（acquire 时 push_off 关本核中断）

注：tty 子系统里全部是 spinlock，没有 sleeplock/mutex；"睡眠"一律靠 sleep(chan, spinlock) 在等待时临时放锁。